Software Requirements Specification (SRS)

Week 3 Mini Project: Employee Analytics Console Application

1. Project Overview

The **Employee Analytics Console Application** is a Java console-based project designed to consolidate all concepts learned during Week 3 of the Java Backend Mastery Roadmap.

The project focuses heavily on **Java 8 functional programming concepts**, including:

- Lambda Expressions & Method References
- Functional Interfaces (Predicate, Consumer, Supplier)
- Stream API & Advanced Stream Operations (filter, map, reduce, collect, sorting, and grouping)
- Optional class for null-safe data retrieval

The application will manage an in-memory collection of employees and provide deep business analytics and reporting features through a menu-driven console interface. The primary objective is to perform meaningful analytics using Java Streams rather than merely storing data.

2. Learning Objectives

By completing this project, you must demonstrate the ability to:

- Write and apply **Lambda Expressions** and **Method References** appropriately.
- Create and utilize standard and custom **Functional Interfaces**.
- Apply Predicate, Consumer, and Supplier interfaces to real-world logic.
- Convert traditional loops (for, while) into elegant **Stream-based solutions**.
- Mastery over Stream methods: filter(), map(), sorted(), collect(), reduce(), and groupingBy().
- Use Optional for safe, exception-free data retrieval.
- Design a clean, modular, object-oriented Java application.
- Confidently explain the underlying mechanics of every stream operation used.

3. Functional Requirements

- **In-Memory Data Store:** The system shall maintain employee records in memory using Java Collections.
- **Employee Attributes:** Each employee record must contain:
    - Employee ID (int / String)
    - Employee Name (String)
    - Department (String)
    - Salary (double)

    - Age (int)
    - Email Address (String)
- **Seed Data:** The application shall initially load at least **15 sample employee records** upon startup to allow meaningful analytics.
- **User Interface:** The application shall provide a clean, readable, menu-driven console interface for execution.

4. Mandatory Features

Basic Features

- **Feature 1 – Display All Employees:** Display every employee record in a well-formatted console table.
- **Feature 6 – Employee Name Extraction:** Transform and display *only* employee names using map().
- **Feature 14 – Employee Lookup by ID:** Search for an employee by ID and wrap the result in an Optional.
- **Feature 15 – Exit Application:** Gracefully terminate the program.

Filtering & Sorting

- **Feature 2 – Find Employees by Department:** Accept a department name from the user and display matching employees using stream filtering.
- **Feature 3 – Find High Salary Employees:** Display employees whose salary exceeds a predefined user threshold.
- **Feature 4 – Sort Employees by Salary:** Allow the user to sort records by salary in both **ascending** and **descending**order.
- **Feature 5 – Sort Employees by Name:** Display employee records sorted alphabetically.

Aggregations & Analytics

- **Feature 7 – Total Salary Expenditure:** Calculate the company's total payroll expenditure using reduce().
- **Feature 8 – Average Salary:** Calculate and display the average salary across the company.
- **Feature 9 – Highest Paid Employee:** Identify and display the details of the employee with the highest salary.
- **Feature 10 – Lowest Paid Employee:** Identify and display the details of the employee with the lowest salary.

Advanced Grouping Analytics

- **Feature 11 – Group Employees by Department:** Display department-wise employee groups using groupingBy().
- **Feature 12 – Employee Count Per Department:** Compute and display the total number of employees in each department.
- **Feature 13 – Average Department Salary:** Compute and display the average salary specifically for each department.

5. Java 8 Checklist Integration

The project must explicitly showcase the following items from the Java 8 roadmap:

1. **Lambda Expressions:** Write multiple lambdas across filtering/mapping operations.
2. **Functional Interfaces:** Utilize standard functional interfaces natively.
3. **Loop Replacement:** Eliminate traditional loops in favor of Streams.
4. **filter() & Predicate:** Use extensively for conditional filtering.
5. **map():** Use for data transformations (e.g., pulling names out of objects).
6. **reduce():** Implement for payroll aggregate calculations.
7. **collect():** Use to transition stream pipelines back into result collections.
8. **sorted():** Apply for custom ordering operations.
9. **groupingBy():** Leverage for complex analytical classification reports.
10. **Optional:** Implement for defensive, safe employee lookup operations.
11. **Method References:** Substitute verbose lambdas with method references (Class::method) where applicable.
12. **Consumer:** Apply for printing/output actions.
13. **Supplier:** Utilize for generating test data or IDs.

6. Suggested Project Structure

To maintain a modular, production-grade architecture, organize your code into the following structure:

- **Employee.java**
  - POJO containing employee attributes, constructors, getters, setters, and an overridden toString() method.
- **EmployeeDataProvider.java**
  - Utility class that hardcodes and returns the initial batch of 15+ sample employee records.
- **EmployeeService.java**
  - The core engine containing all business logic, data processing pipelines, and stream-based analytics methods.
- **Main.java**
  - The presentation layer handling user input, displaying the console menu loop, and interacting with the service layer.
- *Optional:* **Custom Functional Interfaces**
  - Create at least one custom functional interface to solidify your foundational understanding of how @FunctionalInterface works under the hood.

7. Stream Operations Mapping

Ensure your service methods map directly to these specific operations:

- **filter()** $\rightarrow$ Department searches, high-salary conditional filtering.
- **map()** $\rightarrow$ Converting Employee objects into String names or specific numeric ranges.
- **sorted()** $\rightarrow$ Alphabetical sorting and numeric salary sequencing.

- **collect()** $\rightarrow$ Collapsing downstream operations back into Lists, Sets, or Maps.
- **reduce()** $\rightarrow$ Handling calculations like total payroll budget.
- **groupingBy()** $\rightarrow$ Multi-level or single-level department categorization pipelines.

8. Non-Functional Requirements

- **Environment:** Built using **Java 8** or higher.
- **Interface:** Strictly a **console-based** command line application.
- **Code Quality:** Adherence to clean code practices, descriptive naming conventions, and the **DRY (Don't Repeat Yourself)** principle.
- **Design:** High modularity with clear separation of concerns between data provider, service, and UI layers.
- **UX:** Well-aligned, human-readable tabular output layouts with smooth, exception-resistant menu navigation.

9. Sample Console Menu

```
==================================================
      EMPLOYEE ANALYTICS CONSOLE MENU
==================================================
1. Display All Employees
2. Search Employees By Department
3. Show High Salary Employees
4. Sort By Salary (Ascending)
5. Sort By Salary (Descending)
6. Sort By Name
7. Display Employee Names Only
8. Total Salary Expenditure
9. Average Salary
10. Highest Paid Employee
11. Lowest Paid Employee
12. Group Employees By Department
13. Employee Count Per Department
14. Average Salary Per Department
15. Find Employee By ID
16. Exit
==================================================
Enter choice (1-16):
```

10. Deliverables

Your final project submission bundle must include:

1. **Source Code:** Fully compiled, warning-free Java files following the suggested package structure.
2. **README.md:** An explanatory documentation file detailing how to build, run, and interact with the console application.

3. **Output Screenshots:** A dedicated folder containing step-by-step visual proof of each functional feature running in the console.
4. **Git Repository:** Access to a GitHub link showing incremental, meaningful commit histories representing your development lifecycle.

11. Completion Criteria

The project is officially considered complete when:

- Every single item on the **Java 8 Checklist** is naturally integrated.
- All **15 mandatory menu features** work perfectly without runtime crashes.
- Absolutely no standard loops (for/while) are used for querying or data analytics.
- Optional handles missing target lookups seamlessly without raising NullPointerException errors.